

Daily Paper Review: Remember When It Matters — Proactive Memory

Agent for Long-Horizon Agents

Internbot — Automated Research Summary

July 11, 2026

1 Paper Summary

1.1 Problem and Motivation

Wu et al. [1] identify a specific failure mode of long-horizon LLM agents called **behavioral state decay**: during extended multi-step execution, information that *should* shape future actions—task requirements, environment facts, previous attempts, failure diagnoses, open subgoals—**stops influencing the agent’s next decision**, even when the information may still technically exist within the context window. This is critically different from information being *lost*: the key insight is that information can be *present but behaviorally inactive*—it no longer exerts reliable control over agent behavior.

Why Existing Memory Falls Short. Existing memory systems (MemGPT, MemoryBank, Generative Agents, Mem0) focus on *storing* information persistently and *retrieving* relevant records on demand. For active task execution, however, a fundamentally different problem arises: **memory must decide *when* to intervene**. A summarizer asks “what to retain”; this paper asks “whether any retained execution state should become active in the agent’s next decision.” This is the distinction between *reactive* memory (retrieves when queried) and *proactive* memory (autonomously decides whether to inject information, or to remain silent).

The Importance of Silence. Fixed “always-on” memory injection can distract, consume tokens, and hurt performance on some task types. The null intervention (silence) must be an *explicit policy choice*, not a default—the memory agent should actively decide that no intervention is needed, based on the current trajectory state.

1.2 Method

The paper proposes a **Proactive Memory Agent**—a separate process that runs alongside an unmodified action agent, observing the trajectory, maintaining structured execution state, and making a binary intervention decision at each step.

Architecture. The system has two decoupled agents:

1. **Action Agent** (unmodified, e.g., Claude Sonnet 4.5 or Opus 4.6): Interacts with the environment, issues tool calls.
2. **Memory Agent** (separate): Observes a sliding window of $k = 8$ recent messages, maintains the memory bank, and decides whether to inject a targeted reminder into the action agent’s next call.

The memory agent is **plug-and-play**—it does not modify the action agent’s instructions, tools, or decoding procedure.

Memory Bank. Structured as $\mathcal{B}_t = (s_t, \mathcal{K}_t, \mathcal{P}_t)$:

- s_t (**Status**): Private to the memory agent—tracks progress, open issues, unresolved risks. Never shown to the action agent. Provides the memory agent a “working model” of the task.
- $\mathcal{K}_t$ (**Knowledge memories**): Stable facts—task requirements, environment properties, file paths, configuration details, user/tool-verified facts.
- $\mathcal{P}_t$ (**Procedural memories**): Attempts and outcomes—commands that failed, fixes that succeeded, hypotheses ruled out, diagnostic signals.

Each entry has a short identifier, natural-language content, creation time, and access statistics.

Two-Phase Internal Workflow. At each memory timestep:

Phase 1 — Bank Update: $\mathcal{B}_t \sim \pi_M^{\text{edit}}(\cdot \mid x, w_t, \mathcal{B}_{t-1})$, where $w_t = W_k(\tau_{<t}, o_t)$ is the sliding window. The memory agent issues zero or more tool calls: `memory_update_status`, `memory_save_knowledge`, `memory_save_procedural`, `memory_delete`.

Phase 2 — Intervention Decision: $i_t \sim \pi_M^{\text{intervene}}(\cdot \mid x, w_t, \mathcal{B}_t)$, where $i_t \in \{\emptyset, r_t\}$. A non-null intervention r_t is injected as *transient* memory context into the next action-agent call. The intervention is triggered when:

- Requirements are about to be violated
- Environment facts explain the current observation
- Previous attempts should not be repeated
- Diagnoses remain relevant
- Open subgoals are being neglected

Learning the Intervention Policy. For open-weight deployment (Qwen3.5-27B memory agent):

1. **SFT**: Distill trajectories from the prompted Opus 4.6 memory agent, teaching the 27B model to perform bank operations and choose between reminder/silence.
2. **GRPO**: Further calibrate intervention timing using RL. Updates focus on “pivot turns” identified from offline rollouts as likely to affect downstream success. Reward: binary task-level pass/fail.

1.3 Results

Setup. Benchmarks: Terminal-Bench 2.0 (85 containerized terminal tasks—debugging, file editing, command execution) and τ^2 -Bench (278 conversational tool-use tasks across airline/retail/telecom domains). Action agents: Claude Sonnet 4.5 and Opus 4.6. Memory agent: Claude Opus 4.6 (prompted) or Qwen3.5-27B (trained).

Main Results.

- Terminal-Bench 2.0: **+8.3 pp** with Sonnet 4.5 (37.6% $\rightarrow$ 45.9%), +2.4 pp with Opus 4.6 (43.5% $\rightarrow$ 45.9%).
- τ^2 -Bench (weighted avg): **+6.8 pp** with Sonnet 4.5 (55.0% $\rightarrow$ 61.8%), +2.5 pp with Opus 4.6 (66.2% $\rightarrow$ 68.7%).
- Per-domain: airline +10.0 pp, retail +9.6 pp, telecom +2.6 pp.

Gains are larger for weaker action agents but do *not disappear* for stronger ones.

Ablation. On τ^2 -Bench with Sonnet 4.5 (macro average):

- Full memory agent: **64.3%**.
- Full-bank context (passive exposure): 61.5% (−2.8 pp). Maintaining memory is useful, but exposing *all* of it is suboptimal.
- Always inject (no silence): 63.5% (−0.8 pp). Competitive overall but loses on airline, showing silence matters for balanced performance.
- Injection-only (no persistent bank): 61.0% (−3.3 pp). Helps telecom (+11.4 pp) but *hurts* airline (−6.0 pp)—unstable without persistent memory.
- Mem0 (vector+BM25 top-10 retrieval): 62.1% (−2.2 pp). Improves average but fails to improve airline at all—retrieval alone is insufficient without intervention modeling.

Open-Weight Training. Qwen3.5-27B memory agent with Qwen3.5-122B action agent:

- Untrained 27B memory: **hurts** performance (−0.016 avg reward). Naive memory injection is harmful.
- SFT memory: +0.011 avg reward (recovers and modestly improves).
- GRPO memory: **+0.025** avg reward, transfers to held-out Terminal-Bench (+3.5 pp).

1.4 Limitations

Fixed triggering schedule: The memory agent runs at every step rather than learning *when* to be invoked. More selective triggers could reduce overhead.

Memory agent cost: Running Opus 4.6 as the memory agent at every step adds significant inference cost. The open-weight alternative (27B) partially addresses this but doesn’t close the performance gap.

Same-model ceiling: When action agent = Opus 4.6, gains are much smaller (+2.4/+2.5 pp), suggesting diminishing returns when the action agent is already strong.

Domain variability: Telecom consistently shows smaller gains than airline/retail, and injection-only helps telecom while hurting airline—the architecture’s benefit is not uniform.

No joint training: Action and memory agents are never trained jointly. The action agent cannot learn to better *use* the interventions it receives.

Calibration errors: The memory agent sometimes surfaces speculative inferences with too much confidence, repeats information already known, or raises unnecessary concerns.

1.5 Relevance to Our Research

This paper advances our T3 (Continual Learning for Agents) thread by reframing agent memory from a storage/retrieval problem to an **intervention policy problem**. Our previous memory reviews—Memanto [2] (typed semantic memory with information-theoretic retrieval), EvoEmbedding [3] (evolvable representations for agentic memory), EvoArena [4] (tracking memory evolution in dynamic environments)—all focused on *what* to store and *how* to retrieve. The proactive memory agent adds the missing dimension: *whether and when* to inject stored information into the agent’s decision process.

The behavioral state decay concept provides a formal name for a phenomenon our corpus has addressed indirectly. Our Geometry Conflict review [5] showed that gradient interference causes forgetting during training; proactive memory addresses the analogous *inference-time* phenomenon where information is present but inactive. Our Rethinking Continual Experience Internalization review [6] studied how agents internalize experiences; the proactive memory agent’s procedural memory ($\mathcal{P}_t$) is an online instantiation of this—accumulating failed/successful approaches across the task horizon and surfacing them at decision points.

The connection to our RL reviews (T2) is through the GRPO-based intervention policy training. The paper uses binary task-level rewards to calibrate *when* to intervene, connecting to our SAO review [7] (single-rollout RL for agents) and Verification Horizon [8] (reward design for agent tasks). The “pivot turn” identification—finding turns where intervention is most likely to change outcomes—is conceptually related to DelTA’s [9] discriminative token credit assignment, but at the turn level rather than token level.

The plug-and-play architecture has practical implications for our on-policy distillation research (T1). Our UI-MOPD review [10] used platform-specific teachers as behavioral anchors during OPD. The proactive memory agent is conceptually similar—an external behavioral anchor that constrains the action agent without modifying its architecture—but operates at inference time rather than training time. This suggests a spectrum: OPD anchors (training-time, parameter-level), proactive memory (inference-time, context-level), and MIPU validation (post-update, acceptance-level) [11].

2 Follow-up Research Directions

2.1 Direction 1: Training-Time Memory Internalization via Proactive Distillation

Gap. The proactive memory agent operates at inference time, adding latency and cost at every step. If the *types of interventions* that help are predictable (e.g., surfacing requirements before they’re violated, reminding of failed approaches), then these intervention patterns could be *internalized* into the action agent during training, eliminating the need for a separate memory agent at deployment. Our continual experience internalization review [6] showed that agents can internalize experience into parameters, but did not consider internalizing the *timing* of memory retrieval.

Approach. Design a **proactive distillation** pipeline:

1. **Collect intervention trajectories:** Run the proactive memory agent alongside the action agent on training tasks. Record: (a) turns where the memory agent intervened, (b) the content of the intervention, (c) whether the task ultimately succeeded.
2. **Intervention-conditioned SFT:** Fine-tune the action agent on successful trajectories, but at turns where interventions occurred, concatenate the memory injection into the action agent’s input. This teaches the action agent to *expect* and leverage memory-style information at critical turns.

3. **Self-elicitation training:** After intervention-conditioned SFT, train the action agent to *generate its own* memory-style reflections at the same turns where external interventions were previously needed. Use a classifier (trained on intervention/no-intervention labels) to predict when self-elicitation should trigger.

This progressively transfers the memory agent’s intervention policy into the action agent’s own behavior—first teaching it to use interventions, then teaching it to produce them internally.

Feasibility. Intervention trajectories are a byproduct of the existing system. Intervention-conditioned SFT is standard fine-tuning with augmented context. The self-elicitation classifier adds minimal overhead (<1% parameters). Validation: compare action agent alone vs. action agent + external memory vs. action agent with internalized memory on Terminal-Bench. Expected outcome: internalized memory recovers 60–80% of the external memory agent’s gains while eliminating the per-step cost of running a separate model.

2.2 Direction 2: Hierarchical Intervention Policy with Cross-Task Memory Transfer

Gap. The proactive memory agent treats each task independently—the memory bank $\mathcal{B}_t$ is initialized empty at the start of each task and discarded at the end. In real deployments, an agent encounters many similar tasks sequentially (e.g., repeated debugging sessions on the same codebase, recurring customer service patterns). Knowledge gained in one task (“this API always returns errors with header X”, “this customer type always needs action Y”) should transfer to future tasks. This is the classic continual learning challenge but at the memory agent level.

Approach. Design a **hierarchical memory** with cross-task transfer:

1. **Episodic memory** (current system): Per-task $\mathcal{B}_t$, initialized fresh, discarded after completion. Handles within-task behavioral state decay.
2. **Semantic memory** (new): Persistent across tasks. At task completion, the memory agent identifies entries from $\mathcal{K}_t$ and $\mathcal{P}_t$ that are likely transferable (domain-general environment facts, recurring error patterns, common solution strategies) and promotes them to semantic memory.
3. **Transfer policy:** At the start of each new task, the memory agent retrieves relevant entries from semantic memory and seeds the episodic bank. A lightweight classifier determines which semantic memories are relevant to the current task based on task description similarity.
4. **Forgetting prevention:** Semantic memory uses our EvoEmbedding [3] approach (FIFO buffer with bounded re-encoding) to prevent unbounded growth while maintaining coverage of diverse task domains.

This bridges within-task proactive memory (this paper) and cross-task continual learning (our T3 core), creating a two-level system where episodic interventions handle immediate behavioral state decay and semantic transfer handles long-term knowledge accumulation.

Feasibility. The promotion classifier can be trained on labeled task completions (which entries were useful across multiple tasks). FIFO-bounded semantic memory adds fixed storage overhead. Validation: deploy the hierarchical system on a sequence of 50 Terminal-Bench tasks from the same codebase, comparing no-transfer vs. semantic-transfer on later tasks. Expected outcome: semantic transfer improves performance on later tasks by 5–10% as the system accumulates reusable debugging knowledge.

2.3 Direction 3: Intervention Timing as a Reward Shaping Signal for Agent RL

Gap. The proactive memory agent’s GRPO training uses binary task-level rewards (pass/fail). This is sparse—most pivot-turn interventions that prevent failure modes are rewarded only indirectly through eventual task success. Our SAO review [7] showed that token-level value estimation dramatically outperforms step-level for agentic RL. Can the memory agent’s intervention decisions provide a *denser reward signal* for training the action agent itself?

Approach. Use the memory agent’s intervention patterns as **reward shaping** for action agent RL:

1. **Intervention necessity as negative reward:** When the memory agent needs to intervene (injects a reminder), this signals that the action agent was about to make a mistake. Assign a small negative reward $r_{\text{shape}} = -\delta$ at turns where intervention fires. This provides dense, per-turn feedback indicating behavioral state decay.
2. **Successful autonomy as positive reward:** When the memory agent chooses silence (no intervention needed), assign a small positive reward $r_{\text{shape}} = +\delta$. This incentivizes the action agent to maintain behavioral state without external assistance.
3. **Combined RL objective:** Train the action agent with the shaped reward $R_{\text{total}} = R_{\text{task}} + \lambda \sum_t r_{\text{shape}}(t)$, where R_{task} is the sparse task reward and λ controls the shaping strength.
4. **Curriculum:** Start with high λ (memory agent provides strong guidance), gradually anneal to $\lambda = 0$ (action agent operates independently). This creates a natural curriculum from memory-assisted to autonomous operation.

This bridges T2 (RL) and T3 (agent memory): the memory agent becomes a *reward model* that provides dense, per-turn feedback about the action agent’s attentional quality.

Feasibility. Intervention decisions are already computed (they’re the memory agent’s Phase 2 output). The shaped reward adds no inference cost. The curriculum is a simple λ schedule. Validation: compare SAO [7] with sparse task reward vs. SAO with memory-shaped dense reward on Terminal-Bench, using the trained memory agent as the shaping oracle. Expected outcome: memory-shaped reward accelerates training convergence by 2–3 $\times$ and improves final performance by 3–5% by providing the action agent with dense signals about when it’s losing track of execution state.

3 Conclusion

The Proactive Memory Agent reframes agent memory from storage/retrieval to **intervention policy design**. The central insight—that behavioral state decay is an *inference-time* phenomenon requiring active intervention rather than passive exposure—provides the missing link between our training-time continual learning reviews and real-world agent deployment. The ablation revealing that naive memory injection (untrained 27B agent) *hurts* performance while trained intervention *helps* is the most important finding: memory is not inherently beneficial; it becomes beneficial only when coupled with a calibrated policy for *when* and *whether* to surface information. The silence action is not just an efficiency mechanism but an essential component of intervention quality. For our research program, this paper completes a conceptual arc: our training-time methods prevent knowledge from being *lost* (continual learning prevents

forgetting), while the proactive memory agent prevents knowledge from becoming *inactive* (intervention prevents behavioral decay). The three follow-up directions—proactive distillation (internalizing the memory agent’s policy into the action agent), hierarchical cross-task memory (bridging within-task and across-task learning), and intervention-as-reward-shaping (using memory decisions as dense RL signal)—collectively integrate proactive memory into both our training pipelines (T1, T2) and our continual learning architectures (T3).

References

- [1] Wu, Y., Zhang, L., Zhou, Y., et al. “Remember When It Matters: Proactive Memory Agent for Long-Horizon Agents.” *arXiv preprint arXiv:2607.08716*, 2026.
- [2] Memanto Authors. “Memanto: Typed Semantic Memory with Information-Theoretic Retrieval for Long-Horizon Agents.” *arXiv preprint arXiv:2604.22085*, 2026.
- [3] Li, H., et al. “EvoEmbedding: Evolvable Representations for Long-Context Retrieval and Agentic Memory.” *arXiv preprint arXiv:2606.21649*, 2026.
- [4] EvoArena Authors. “EvoArena: Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments.” *arXiv preprint arXiv:2606.13681*, 2026.
- [5] Geometry Conflict Authors. “Geometry Conflict: Explaining and Controlling Forgetting in LLM Continual Post-Training.” *arXiv preprint arXiv:2605.09608*, 2026.
- [6] Rethinking Authors. “Rethinking Continual Experience Internalization for Self-Evolving LLM Agents.” *arXiv preprint arXiv:2606.04703*, 2026.
- [7] Hou, Z., et al. “Single-Rollout Asynchronous Optimization for Agentic Reinforcement Learning.” *arXiv preprint arXiv:2607.07508*, 2026.
- [8] Verification Horizon Authors. “The Verification Horizon: No Silver Bullet for Coding Agent Rewards.” *arXiv preprint arXiv:2606.26300*, 2026.
- [9] DelTA Authors. “DelTA: Discriminative Token Credit Assignment for Reinforcement Learning from Verifiable Rewards.” *arXiv preprint arXiv:2605.21467*, 2026.
- [10] Lian, N., et al. “UI-MOPD: Multi-Platform On-Policy Distillation for Continual GUI Agent Learning.” *arXiv preprint arXiv:2607.04425*, 2026.
- [11] Liang, J., et al. “The Mirage of Optimizing Training Policies: Monotonic Inference Policies as the Real Objective for LLM Reinforcement Learning.” *arXiv preprint arXiv:2606.29526*, 2026.